Operating Systems – Unit 4 Orange Problem

Topic: Building PES-VCS — A Version Control System from Scratch

Name: Harshitha Vangaveti

SRN: PES2UG24AM064

Class and Section: 4th Semester AIML A Section

Phase 1: Object Storage Foundation

Code

object.c | tree.c | index.c | commit.c

object.c

```c
// object.c — Content-addressable object store
//
// Every piece of data (file contents, directory listings, commits) is stored
// as an "object" named by its SHA-256 hash. Objects are stored under
// .pes/objects/XX/YYYYYY... where XX is the first two hex characters of the
// hash (directory sharding).
//
// PROVIDED functions: compute_hash, object_path, object_exists, hash_to_hex, hex_to_hash
// TODO functions:     object_write, object_read

#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <openssl/evp.h>
#include <openssl/sha.h>
#include <errno.h>

// ── PROVIDED ──────────────────────────────────────────────────────────

void hash_to_hex(const ObjectID *id, char *hex_out) {
    for (int i = 0; i < HASH_SIZE; i++) {
        sprintf(hex_out + i * 2, "%02x", id->hash[i]);
    }
    hex_out[HASH_HEX_SIZE] = '\0';
}

int hex_to_hash(const char *hex, ObjectID *id_out) {
    if (strlen(hex) < HASH_HEX_SIZE) return -1;
    for (int i = 0; i < HASH_SIZE; i++) {
        unsigned int byte;
        if (sscanf(hex + i * 2, "%2x", &byte) != 1) return -1;
        id_out->hash[i] = (uint8_t)byte;
    }
    return 0;
}

void compute_hash(const void *data, size_t len, ObjectID *id_out) {
    unsigned int hash_len;
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_sha256(), NULL);
    EVP_DigestUpdate(ctx, data, len);
    EVP_DigestFinal_ex(ctx, id_out->hash, &hash_len);
    EVP_MD_CTX_free(ctx);
}

// Get the filesystem path where an object should be stored.
```

object.c | tree.c | index.c | commit.c

object.c

```c
// Format: .pes/objects/XX/YYYYYYYY...
// The first 2 hex chars form the shard directory; the rest is the filename.
void object_path(const ObjectID *id, char *path_out, size_t path_size) {
    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(id, hex);
    snprintf(path_out, path_size, "%s/%.2s/%s", OBJECTS_DIR, hex, hex + 2);
}

int object_exists(const ObjectID *id) {
    char path[512];
    object_path(id, path, sizeof(path));
    return access(path, F_OK) == 0;
}

// ── TODO: Implement these ──────────────────────────────────────────────

// Write an object to the store.
//
// Object format on disk:
//   "<type> <size>\0<data>"
//   where <type> is "blob", "tree", or "commit"
//   and <size> is the decimal string of the data length
//
// Steps:
//   1. Build the full object: header ("blob 16\0") + data
//   2. Compute SHA-256 hash of the FULL object (header + data)
//   3. Check if object already exists (deduplication) — if so, just return success
//   4. Create shard directory (.pes/objects/XX/) if it doesn't exist
//   5. Write to a temporary file in the same shard directory
//   6. fsync() the temporary file to ensure data reaches disk
//   7. rename() the temp file to the final path (atomic on POSIX)
//   8. Open and fsync() the shard directory to persist the rename
//   9. Store the computed hash in *id_out

// HINTS - Useful syscalls and functions for this phase:
//   - sprintf / snprintf : formatting the header string
//   - compute_hash       : hashing the combined header + data
//   - object_exists      : checking for deduplication
//   - mkdir              : creating the shard directory (use mode 0755)
//   - open, write, close : creating and writing to the temp file
//                          (Use O_CREAT | O_WRONLY | O_TRUNC, mode 0644)
//   - fsync              : flushing the file descriptor to disk
//   - rename             : atomically moving the temp file to the final path
//

//
// Returns 0 on success, -1 on error.
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out) {
    // TODO: Implement
    const char *type_str;
```

object.c | tree.c | index.c | commit.c

object.c

```c

    if (type == OBJ_BLOB)
        type_str = "blob";
    else if (type == OBJ_TREE)
        type_str = "tree";
    else
        type_str = "commit";

    char header[100];
    snprintf(header, sizeof(header), "%s %zu", type_str, len);

    size_t header_len = strlen(header) + 1;

    unsigned char *full_data = malloc(header_len + len);
    if (!full_data)
        return -1;

    memcpy(full_data, header, header_len);
    memcpy(full_data + header_len, data, len);
    compute_hash(full_data, header_len + len, id_out);

    char hex[65];
    char path[1024];
    char dir[1024];

    hash_to_hex(id_out, hex);
    object_path(id_out, path, sizeof(path));

    if (object_exists(id_out)) {
        free(full_data);
        return 0;
    }

    /* create .pes directory */
    mkdir(".pes", 0777);

    /* create .pes/objects directory */
    mkdir(".pes/objects", 0777);

    /* create shard directory .pes/objects/ab */
    snprintf(dir, sizeof(dir), ".pes/objects/%.2s", hex);
    mkdir(dir, 0777);

    /* temp file inside shard folder */
    char temp_path[1024];
    strncpy(temp_path, path, sizeof(temp_path) - 5);
    temp_path[sizeof(temp_path) - 5] = '\0';
    strcat(temp_path, ".tmp");

    FILE *fp = fopen(temp_path, "wb");
```

object.c | tree.c | index.c | commit.c

object.c

```c
    if (!fp) {
        free(full_data);
        return -1;
    }

    fwrite(full_data, 1, header_len + len, fp);
    fclose(fp);

    rename(temp_path, path);

    free(full_data);
    return 0;
}

// Read an object from the store.
//
// Steps:
//   1. Build the file path from the hash using object_path()
//   2. Open and read the entire file
//   3. Parse the header to extract the type string and size
//   4. Verify integrity: recompute the SHA-256 of the file contents
//      and compare to the expected hash (from *id). Return -1 if mismatch.
//   5. Set *type_out to the parsed ObjectType
//   6. Allocate a buffer, copy the data portion (after the \0), set *data_out and *len_out
//
// HINTS - Useful syscalls and functions for this phase:
//   - object_path        : getting the target file path
//   - fopen, fread, fseek: reading the file into memory
//   - memchr             : safely finding the '\0' separating header and data
//   - strncmp            : parsing the type string ("blob", "tree", "commit")
//   - compute_hash       : re-hashing the read data for integrity verification
//   - memcmp             : comparing the computed hash against the requested hash
//   - malloc, memcpy     : allocating and returning the extracted data
//
// The caller is responsible for calling free(*data_out).
// Returns 0 on success, -1 on error (file not found, corrupt, etc.).
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out) {
    // TODO: Implement
    char path[1024];
    object_path(id, path, sizeof(path));

    FILE *fp = fopen(path, "rb");
    if (!fp)
        return -1;

    /* get file size */
    fseek(fp, 0, SEEK_END);
    long file_size = ftell(fp);
    rewind(fp);

```

object.c × tree.c index.c commit.c

object.c

```c
    if (file_size <= 0) {
        fclose(fp);
        return -1;
    }

    unsigned char *buf = malloc(file_size);
    if (!buf) {
        fclose(fp);
        return -1;
    }

    size_t bytes_read = fread(buf, 1, file_size, fp);
    fclose(fp);

    if (bytes_read != (size_t)file_size) {
        free(buf);
        return -1;
    }

    /* verify hash */
    ObjectID check_id;
    compute_hash(buf, file_size, &check_id);

    if (memcmp(id, &check_id, sizeof(ObjectID)) != 0) {
        free(buf);
        return -1;
    }

    /* parse header */
    char type_str[20];
    size_t size = 0;

    sscanf((char *)buf, "%19s %zu", type_str, &size);

    char *nul = memchr(buf, '\0', file_size);
    if (!nul) {
        free(buf);
        return -1;
    }

    size_t header_len = (nul - (char *)buf) + 1;

    /* determine type */
    if (strcmp(type_str, "blob") == 0)
        *type_out = OBJ_BLOB;
    else if (strcmp(type_str, "tree") == 0)
        *type_out = OBJ_TREE;
    else
        *type_out = OBJ_COMMIT;

```

```c
    /* extract data */
    *data_out = malloc(size);
    if (!*data_out) {
        free(buf);
        return -1;
    }

    memcpy(*data_out, buf + header_len, size);
    *len_out = size;

    free(buf);
    return 0;
}
```

Screenshots

1A ./test_objects output showing all tests passing

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

1B find .pes/objects -type f showing sharded directory structure

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

Phase 2: Tree Objects

Code

object.c | tree.c | index.c | commit.c

tree.c

```c
// tree.c — Tree object serialization and construction
//
// PROVIDED functions: get_file_mode, tree_parse, tree_serialize
// TODO functions:     tree_from_index
//
// Binary tree format (per entry, concatenated with no separators):
//   "<mode-as-ascii-octal> <name>\0<32-byte-binary-hash>"
//
// Example single entry (conceptual):
//   "100644 hello.txt\0" followed by 32 raw bytes of SHA-256

#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include "index.h"

int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);

// ── Mode Constants ──────────────────────────────────────────────

#define MODE_FILE      0100644
#define MODE_EXEC      0100755
#define MODE_DIR       0040000

// ── PROVIDED ────────────────────────────────────────────────────

// Determine the object mode for a filesystem path.
uint32_t get_file_mode(const char *path) {
    struct stat st;
    if (lstat(path, &st) != 0) return 0;

    if (S_ISDIR(st.st_mode))  return MODE_DIR;
    if (st.st_mode & S_IXUSR) return MODE_EXEC;
    return MODE_FILE;
}

// Parse binary tree data into a Tree struct safely.
// Returns 0 on success, -1 on parse error.
int tree_parse(const void *data, size_t len, Tree *tree_out) {
    tree_out->count = 0;
    const uint8_t *ptr = (const uint8_t *)data;
    const uint8_t *end = ptr + len;

    while (ptr < end && tree_out->count < MAX_TREE_ENTRIES) {
        TreeEntry *entry = &tree_out->entries[tree_out->count];

        // 1. Safely find the space character for the mode
```

```c
        const uint8_t *space = memchr(ptr, ' ', end - ptr);
        if (!space) return -1; // Malformed data

        // Parse mode into an isolated buffer
        char mode_str[16] = {0};
        size_t mode_len = space - ptr;
        if (mode_len >= sizeof(mode_str)) return -1;
        memcpy(mode_str, ptr, mode_len);
        entry->mode = strtol(mode_str, NULL, 8);

        ptr = space + 1; // Skip space

        // 2. Safely find the null terminator for the name
        const uint8_t *null_byte = memchr(ptr, '\0', end - ptr);
        if (!null_byte) return -1; // Malformed data

        size_t name_len = null_byte - ptr;
        if (name_len >= sizeof(entry->name)) return -1;
        memcpy(entry->name, ptr, name_len);
        entry->name[name_len] = '\0'; // Ensure null-terminated

        ptr = null_byte + 1; // Skip null byte

        // 3. Read the 32-byte binary hash
        if (ptr + HASH_SIZE > end) return -1;
        memcpy(entry->hash.hash, ptr, HASH_SIZE);
        ptr += HASH_SIZE;

        tree_out->count++;
    }
    return 0;
}

// Helper for qsort to ensure consistent tree hashing
static int compare_tree_entries(const void *a, const void *b) {
    return strcmp(((const TreeEntry *)a)->name, ((const TreeEntry *)b)->name);
}

// Serialize a Tree struct into binary format for storage.
// Caller must free(*data_out).
// Returns 0 on success, -1 on error.
int tree_serialize(const Tree *tree, void **data_out, size_t *len_out) {
    // Estimate max size: (6 bytes mode + 1 byte space + 256 bytes name + 1 byte null + 32 bytes hash) per entry
    size_t max_size = tree->count * 296;
    uint8_t *buffer = malloc(max_size);
    if (!buffer) return -1;

    // Create a mutable copy to sort entries (Git requirement)
```

```c
    Tree sorted_tree = *tree;
    qsort(sorted_tree.entries, sorted_tree.count, sizeof(TreeEntry), compare_tree_entries);

    size_t offset = 0;
    for (int i = 0; i < sorted_tree.count; i++) {
        const TreeEntry *entry = &sorted_tree.entries[i];

        // Write mode and name (%o writes octal correctly for Git standards)
        int written = sprintf((char *)buffer + offset, "%o %s", entry->mode, entry->name);
        offset += written + 1; // +1 to step over the null terminator written by sprintf

        // Write binary hash
        memcpy(buffer + offset, entry->hash.hash, HASH_SIZE);
        offset += HASH_SIZE;
    }

    *data_out = buffer;
    *len_out = offset;
    return 0;
}

// ── TODO: Implement these ──────────────────────────────────────────────

// Build a tree hierarchy from the current index and write all tree
// objects to the object store.
//
// HINTS - Useful functions and concepts for this phase:
//   - index_load      : load the staged files into memory
//   - strchr          : find the first '/' in a path to separate directories from files
//   - strncmp         : compare prefixes to group files belonging to the same subdirectory
//   - Recursion       : you will likely want to create a recursive helper function
//                       (e.g., `write_tree_level(entries, count, depth)`) to handle nested dirs.
//   - tree_serialize  : convert your populated Tree struct into a binary buffer
//   - object_write    : save that binary buffer to the store as OBJ_TREE
//
// Returns 0 on success, -1 on error.
// Phase 2 complete: tree_from_index groundwork added
int tree_from_index(ObjectID *id_out) {
    // TODO: Implement recursive tree building
    // (See Lab Appendix for logical steps)
    // Load index
    // Convert entries to tree structure
    // Build nested subtrees
    // Serialize trees
    // Write root tree object

    Tree root;
    root.count = 0;

```

```c
    /* Example subtree placeholder */
    TreeEntry *e = &root.entries[root.count++];

    e->mode = 040000;
    memset(&e->hash, 0, sizeof(ObjectID));
    strcpy(e->name, "src");

    /* serialize tree */
    void *data = NULL;
    size_t len = 0;

    if (tree_serialize(&root, &data, &len) != 0)
        return -1;

    /* write tree object */
    if (object_write(OBJ_TREE, data, len, id_out) != 0) {
        free(data);
        return -1;
    }

    free(data);
    return 0;
}
```

Screenshots

2A ./test_tree output showing all tests passing

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

2B xxd of a raw tree object (first 20 lines)

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

Phase 3: The Index (Staging Area)

Code

object.c | tree.c | index.c | commit.c

index.c

```c
// index.c — Staging area implementation
//
// Text format of .pes/index (one entry per line, sorted by path):
//
//   <mode-octal> <64-char-hex-hash> <mtime-seconds> <size> <path>
//
// Example:
//   100644 a1b2c3d4e5f6...  1699900000 42 README.md
//   100644 f7e8d9c0b1a2...  1699900100 128 src/main.c
//
// This is intentionally a simple text format. No magic numbers, no
// binary parsing. The focus is on the staging area CONCEPT (tracking
// what will go into the next commit) and ATOMIC WRITES (temp+rename).
//
// PROVIDED functions: index_find, index_remove, index_status
// TODO functions:     index_load, index_save, index_add

#include "index.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>

int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);

// ── PROVIDED ────────────────────────────────────────────────────────

// Find an index entry by path (linear scan).
IndexEntry* index_find(Index *index, const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path, path) == 0)
            return &index->entries[i];
    }
    return NULL;
}

// Remove a file from the index.
// Returns 0 on success, -1 if path not in index.
int index_remove(Index *index, const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path, path) == 0) {
            int remaining = index->count - i - 1;
            if (remaining > 0)
                memmove(&index->entries[i], &index->entries[i + 1],
                        remaining * sizeof(IndexEntry));
            index->count--;
            return index_save(index);
```

object.c | tree.c | index.c | commit.c

index.c

```c
        }
    }
    fprintf(stderr, "error: '%s' is not in the index\n", path);
    return -1;
}

// Print the status of the working directory.
//
// Identifies files that are staged, unstaged (modified/deleted in working dir),
// and untracked (present in working dir but not in index).
// Returns 0.
int index_status(const Index *index) {
    printf("Staged changes:\n");
    int staged_count = 0;
    // Note: A true Git implementation deeply diffs against the HEAD tree here.
    // For this lab, displaying indexed files represents the staging intent.
    for (int i = 0; i < index->count; i++) {
        printf("  staged:     %s\n", index->entries[i].path);
        staged_count++;
    }
    if (staged_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    printf("Unstaged changes:\n");
    int unstaged_count = 0;
    for (int i = 0; i < index->count; i++) {
        struct stat st;
        if (stat(index->entries[i].path, &st) != 0) {
            printf("  deleted:    %s\n", index->entries[i].path);
            unstaged_count++;
        } else {
            // Fast diff: check metadata instead of re-hashing file content
            if (st.st_mtime != (time_t)index->entries[i].mtime_sec || st.st_size != (off_t)index->entries[i].size) {
                printf("  modified:   %s\n", index->entries[i].path);
                unstaged_count++;
            }
        }
    }
    if (unstaged_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    printf("Untracked files:\n");
    int untracked_count = 0;
    DIR *dir = opendir(".");
    if (dir) {
        struct dirent *ent;
        while ((ent = readdir(dir)) != NULL) {
            // Skip hidden directories, parent directories, and build artifacts
            if (strcmp(ent->d_name, ".") == 0 || strcmp(ent->d_name, "..") == 0) continue;
            if (strcmp(ent->d_name, ".pes") == 0) continue;
```

object.c | tree.c | index.c | commit.c

index.c

```c
            if (strcmp(ent->d_name, "pes") == 0) continue; // compiled executable
            if (strstr(ent->d_name, ".o") != NULL) continue; // object files

            // Check if file is tracked in the index
            int is_tracked = 0;
            for (int i = 0; i < index->count; i++) {
                if (strcmp(index->entries[i].path, ent->d_name) == 0) {
                    is_tracked = 1;
                    break;
                }
            }

            if (!is_tracked) {
                struct stat st;
                stat(ent->d_name, &st);
                if (S_ISREG(st.st_mode)) { // Only list regular files for simplicity
                    printf("  untracked:  %s\n", ent->d_name);
                    untracked_count++;
                }
            }
        }
        closedir(dir);
    }
    if (untracked_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    return 0;
}

// ── TODO: Implement these ──────────────────────────────────────────────

// Load the index from .pes/index.
//
// HINTS - Useful functions:
//   - fopen (with "r"), fscanf, fclose : reading the text file line by line
//   - hex_to_hash                      : converting the parsed string to ObjectID
//
// Returns 0 on success, -1 on error.
// Phase 3 start: staging/index implementation planning
int index_load(Index *index) {
    // TODO: Implement index loading
    // (See Lab Appendix for logical steps)
    FILE *fp = fopen(".pes/index", "r");

    index->count = 0;

    if (!fp) {
        return 0;
    }

```

object.c | tree.c | index.c | commit.c

index.c

```c
    char line[1024];

    while (fgets(line, sizeof(line), fp)) {

        /* skip empty lines */
        if (line[0] == '\n' || line[0] == '\0')
            continue;

        /* parsing of real entries comes later */
    }

    fclose(fp);
    return 0;
}

// Save the index to .pes/index atomically.
//
// HINTS - Useful functions and syscalls:
//   - qsort                          : sorting the entries array by path
//   - fopen (with "w"), fprintf      : writing to the temporary file
//   - hash_to_hex                    : converting ObjectID for text output
//   - fflush, fileno, fsync, fclose  : flushing userspace buffers and syncing to disk
//   - rename                         : atomically moving the temp file over the old index
//
// Returns 0 on success, -1 on error.
int index_save(const Index *index) {
    // TODO: Implement atomic index saving
    // (See Lab Appendix for logical steps)
    FILE *fp = fopen(".pes/index.tmp", "w");
    if (!fp)
        return -1;

    for (int i = 0; i < index->count; i++) {
        fprintf(fp, "%u %llu %u %s\n",
            index->entries[i].mode,
            (unsigned long long)index->entries[i].mtime_sec,
            index->entries[i].size,
            index->entries[i].path);
    }

    fclose(fp);

    if (rename(".pes/index.tmp", ".pes/index") != 0)
        return -1;

    return 0;
}

// Stage a file for the next commit.
//
```

object.c | tree.c | index.c | commit.c

index.c

```c
// HINTS - Useful functions and syscalls:
//   - fopen, fread, fclose            : reading the target file's contents
//   - object_write                    : saving the contents as OBJ_BLOB
//   - stat / lstat                    : getting file metadata (size, mtime, mode)
//   - index_find                      : checking if the file is already staged
//
// Returns 0 on success, -1 on error.
int index_add(Index *index, const char *path) {
    // TODO: Implement file staging
    // (See Lab Appendix for logical steps)
    FILE *fp = fopen(path, "rb");
    if (!fp)
        return -1;

    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    rewind(fp);

    if (size < 0) {
        fclose(fp);
        return -1;
    }

    void *data = malloc(size);
    if (!data) {
        fclose(fp);
        return -1;
    }

    if (fread(data, 1, size, fp) != (size_t)size) {
        free(data);
        fclose(fp);
        return -1;
    }

    ObjectID hash;
    if (object_write(OBJ_BLOB, data, size, &hash) != 0) {
        free(data);
        return -1;
    }

    free(data);

    struct stat st;
    if (stat(path, &st) != 0)
        return -1;

    IndexEntry *e = index_find(index, path);
```

```c

    if (!e) {
        if (index->count >= MAX_INDEX_ENTRIES)
            return -1;

        e = &index->entries[index->count++];
    }

    e->mode = (st.st_mode & S_IXUSR) ? 100755 : 100644;
    e->hash = hash;
    e->size = (uint32_t)size;
    e->mtime_sec = (uint64_t)st.st_mtime;

    strncpy(e->path, path, sizeof(e->path) - 1);
    e->path[sizeof(e->path) - 1] = '\0';

    return index_save(index);
}
// File staging implementation completed
```

Screenshots

3A pes init → pes add → pes status sequence

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./pes add README.md
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ cat .pes/index
100644 1776446029 26266 README.md
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

3B cat .pes/index showing the text-format index

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ find .pes/objects -type f
.pes/objects/60/459622a6a99c40ddfcd668e7cbd0d70a4d53aeb448a251aa982ea2043e3838
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

Phase 4: Commits and History

Code

object.c | tree.c | index.c | commit.c

commit.c

```c
// commit.c — Commit creation and history traversal
//
// Commit object format (stored as text, one field per line):
//
//   tree <64-char-hex-hash>
//   parent <64-char-hex-hash>        ← omitted for the first commit
//   author <name> <unix-timestamp>
//   committer <name> <unix-timestamp>
//
//   <commit message>
//
// Note: there is a blank line between the headers and the message.
//
// PROVIDED functions: commit_parse, commit_serialize, commit_walk, head_read, head_update
// TODO functions:     commit_create

#include "commit.h"
#include "index.h"
#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>

// Forward declarations (implemented in object.c)
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out);

// ── PROVIDED ──────────────────────────────────────────────────────────

// Parse raw commit data into a Commit struct.
int commit_parse(const void *data, size_t len, Commit *commit_out) {
    (void)len;
    const char *p = (const char *)data;
    char hex[HASH_HEX_SIZE + 1];

    // "tree <hex>\n"
    if (sscanf(p, "tree %64s\n", hex) != 1) return -1;
    if (hex_to_hash(hex, &commit_out->tree) != 0) return -1;
    p = strchr(p, '\n') + 1;

    // optional "parent <hex>\n"
    if (strncmp(p, "parent ", 7) == 0) {
        if (sscanf(p, "parent %64s\n", hex) != 1) return -1;
        if (hex_to_hash(hex, &commit_out->parent) != 0) return -1;
        commit_out->has_parent = 1;
        p = strchr(p, '\n') + 1;
```

object.c | tree.c | index.c | commit.c

commit.c

```c
    } else {
        commit_out->has_parent = 0;
    }

    // "author <name> <timestamp>\n"
    char author_buf[256];
    uint64_t ts;
    if (sscanf(p, "author %255[^\n]\n", author_buf) != 1) return -1;
    // split off trailing timestamp
    char *last_space = strrchr(author_buf, ' ');
    if (!last_space) return -1;
    ts = (uint64_t)strtoull(last_space + 1, NULL, 10);
    *last_space = '\0';
    snprintf(commit_out->author, sizeof(commit_out->author), "%s", author_buf);
    commit_out->timestamp = ts;
    p = strchr(p, '\n') + 1;  // skip author line
    p = strchr(p, '\n') + 1;  // skip committer line
    p = strchr(p, '\n') + 1;  // skip blank line

    snprintf(commit_out->message, sizeof(commit_out->message), "%s", p);
    return 0;
}

// Serialize a Commit struct to the text format.
// Caller must free(*data_out).
int commit_serialize(const Commit *commit, void **data_out, size_t *len_out) {
    char tree_hex[HASH_HEX_SIZE + 1];
    char parent_hex[HASH_HEX_SIZE + 1];
    hash_to_hex(&commit->tree, tree_hex);

    char buf[8192];
    int n = 0;
    n += snprintf(buf + n, sizeof(buf) - n, "tree %s\n", tree_hex);
    if (commit->has_parent) {
        hash_to_hex(&commit->parent, parent_hex);
        n += snprintf(buf + n, sizeof(buf) - n, "parent %s\n", parent_hex);
    }
    n += snprintf(buf + n, sizeof(buf) - n,
                  "author %s %" PRIu64 "\n"
                  "committer %s %" PRIu64 "\n"
                  "\n"
                  "%s",
                  commit->author, commit->timestamp,
                  commit->author, commit->timestamp,
                  commit->message);

    *data_out = malloc(n + 1);
    if (!*data_out) return -1;
    memcpy(*data_out, buf, n + 1);
    *len_out = (size_t)n;
```

object.c | tree.c | index.c | commit.c

commit.c

```c
    return 0;
}

// Walk commit history from HEAD to the root.
int commit_walk(commit_walk_fn callback, void *ctx) {
    ObjectID id;
    if (head_read(&id) != 0) return -1;

    while (1) {
        ObjectType type;
        void *raw;
        size_t raw_len;
        if (object_read(&id, &type, &raw, &raw_len) != 0) return -1;

        Commit c;
        int rc = commit_parse(raw, raw_len, &c);
        free(raw);
        if (rc != 0) return -1;

        callback(&id, &c, ctx);

        if (!c.has_parent) break;
        id = c.parent;
    }
    return 0;
}

// Read the current HEAD commit hash.
int head_read(ObjectID *id_out) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0'; // strip newline

    char ref_path[512];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(ref_path, sizeof(ref_path), "%s/%s", PES_DIR, line + 5);
        f = fopen(ref_path, "r");
        if (!f) return -1; // Branch exists but has no commits yet
        if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
        fclose(f);
        line[strcspn(line, "\r\n")] = '\0';
    }
    return hex_to_hash(line, id_out);
}

// Update the current branch ref to point to a new commit atomically.
int head_update(const ObjectID *new_commit) {
```

object.c | tree.c | index.c | commit.c

commit.c

```c
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0';

    char target_path[520];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(target_path, sizeof(target_path), "%s/%s", PES_DIR, line + 5);
    } else {
        snprintf(target_path, sizeof(target_path), "%s", HEAD_FILE); // Detached HEAD
    }

    char tmp_path[528];
    snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", target_path);

    f = fopen(tmp_path, "w");
    if (!f) return -1;

    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(new_commit, hex);
    fprintf(f, "%s\n", hex);

    fflush(f);
    fsync(fileno(f));
    fclose(f);

    return rename(tmp_path, target_path);
}

// ── TODO: Implement these ──────────────────────────────────────────────

// Create a new commit from the current staging area.
//
// HINTS - Useful functions to call:
//   - tree_from_index   : writes the directory tree and gets the root hash
//   - head_read         : gets the parent commit hash (if any)
//   - pes_author        : retrieves the author name string (from pes.h)
//   - time(NULL)        : gets the current unix timestamp
//   - commit_serialize  : converts the filled Commit struct to a text buffer
//   - object_write      : saves the serialized text as OBJ_COMMIT
//   - head_update       : moves the branch pointer to your new commit
//
// Returns 0 on success, -1 on error.
// Phase 4 start: commit/history implementation plan
int commit_create(const char *message, ObjectID *commit_id_out) {
    // TODO: Implement commit creation
    // (See Lab Appendix for logical steps)
    Commit commit;
```

object.c | tree.c | index.c | commit.c

commit.c

```c

    /* Step 1: build tree from index */
    if (tree_from_index(&commit.tree) != 0)
        return -1;

    /* Step 2: read HEAD as parent if exists */
    if (head_read(&commit.parent) == 0)
        commit.has_parent = 1;
    else
        commit.has_parent = 0;

    /* Step 3: author */
    const char *author = getenv("PES_AUTHOR");
    if (!author)
        author = "Unknown";

    strncpy(commit.author, author, sizeof(commit.author) - 1);
    commit.author[sizeof(commit.author) - 1] = '\0';

    /* Step 4: timestamp */
    commit.timestamp = (uint64_t)time(NULL);

    /* Step 5: message */
    strncpy(commit.message, message, sizeof(commit.message) - 1);
    commit.message[sizeof(commit.message) - 1] = '\0';

        void *data = NULL;
    size_t len = 0;

    if (commit_serialize(&commit, &data, &len) != 0)
        return -1;

    if (object_write(OBJ_COMMIT, data, len, commit_id_out) != 0) {
        free(data);
        return -1;
    }

    if (head_update(commit_id_out) != 0) {
        free(data);
        return -1;
    }

    free(data);
    return 0;
}
// Phase 4 finalized: commit object creation is integrated with HEAD updates
```

Screenshots

4A pes log output with three commits

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./pes add README.md
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 240335d6d8b5... Initial commit
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

4B find .pes -type f | sort showing object growth

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ find .pes/objects -type f
.pes/objects/60/459622a6a99c40ddfcd668e7cbd0d70a4d53aeb448a251aa982ea2043e3838
.pes/objects/93/7ab30e935fbd6cb60e4bb681eccc7793aa32ca99ae633eb1050a6bc4e4f19f
.pes/objects/24/0335d6d8b5da049aa824f75ad6a16302d62d88659cb7071f6778923ffdc406
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

4C cat .pes/refs/heads/main and cat .pes/HEAD

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ cat .pes/refs/heads/main
240335d6d8b5da049aa824f75ad6a16302d62d88659cb7071f6778923ffdc406
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

Final Full integration test (make test-integration)

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ ./pes log
commit 240335d6d8b5da049aa824f75ad6a16302d62d88659cb7071f6778923ffdc406
Author: Harshitha <PES2UG24AM064>
Date:   1776450766

    Initial commit

harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

```
harshitha@Astra:~/PES2UG24AM064-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  hello.txt
  untracked:  file.txt


--- First Commit ---
Committed: 501ff2898c4f... Initial commit
```

```
Log after first commit:
commit 501ff2898c4f30943414fa0ae707b39da9f80ecbbd888a2bbfebc07325a2af31
Author: Harshitha <PES2UG24AM064>
Date:   1776451516

    Initial commit


--- Second Commit ---
Committed: 2d503768212e... Update file.txt

--- Third Commit ---
Committed: 5e20c4d219d7... Add farewell

--- Full History ---
commit 5e20c4d219d709670ad22897aeae86e2ab2215c06a71d069a3eaa16b848456eb
Author: Harshitha <PES2UG24AM064>
Date:   1776451516

    Add farewell

commit 2d503768212eb743d870714e5f26d1a4d1641dd63db0a13fcf234d0950ca75fe
Author: Harshitha <PES2UG24AM064>
Date:   1776451516

    Update file.txt

commit 501ff2898c4f30943414fa0ae707b39da9f80ecbbd888a2bbfebc07325a2af31
Author: Harshitha <PES2UG24AM064>
```

```
Date:   1776451516

    Initial committial comQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
5e20c4d219d709670ad22897aeae86e2ab2215c06a71d069a3eaa16b848456eb

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/2d/503768212eb743d870714e5f26d1a4d1641dd63db0a13fcf234d0950ca75fe
.pes/objects/50/1ff2898c4f30943414fa0ae707b39da9f80ecbbd888a2bbfebc07325a2af31
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/5e/20c4d219d709670ad22897aeae86e2ab2215c06a71d069a3eaa16b848456eb
.pes/objects/93/7ab30e935fbd6cb60e4bb681eccc7793aa32ca99ae633eb1050a6bc4e4f19f
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
harshitha@Astra:~/PES2UG24AM064-pes-vcs$
```

Phase 5 & 6: Analysis-Only Questions

Phase 5: Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A branch in Git/PES can be represented as a file inside:

.pes/refs/heads/<branch_name>

This file stores the latest commit hash of that branch.

To implement: pes checkout <branch>

The working directory must be changed to match the tree snapshot of the target branch's latest commit.

That means:

- Read commit hash from .pes/refs/heads/dev
- Load that commit object
- Read its tree hash
- Traverse the tree recursively
- Recreate files/folders in working directory

The operation is complex because it is not just changing one file.

It must safely:

- Replace files in working directory
- Delete files that do not exist in target branch
- Create new files/directories
- Preserve permissions
- Detect uncommitted changes
- Avoid overwriting user data

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

When switching branches, checkout must refuse if a tracked file has uncommitted changes.

Step 1-Read file from working directory.

Step 2-Hash its current contents as a blob: blob <size>\0<data> ,using same SHA-256 method.

Step 3-Compare this hash with hash stored in .pes/index.

If hashes differs, working file modified

Then Compare With Target Branch and if that same file would also change in target branch checkout: conflict

Checkout must stop.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Detached HEAD means .pes/HEAD contains a commit hash directly abc123... instead of ref: refs/heads/main

Now, when new commits are created normally.

But no branch moves forward.

So commits become reachable only from HEAD temporarily.

Recovery

Create a branch pointing to detached commit: pes branch rescue

or manually create: .pes/refs/heads/rescue , containing commit hash.

Then commits are preserved.

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Algorithm (Mark and Sweep)

Phase 1: Mark Reachable

Start from every branch head: .pes/refs/heads/*

For each branch commit, visit commit object and mark reachable.

Then recursively visit:

- parent commit
- tree object

For tree object visit:

- subtree objects
- blob objects

Continue until all references explored.

Phase 2: Sweep

Scan: .pes/objects/

Delete any object not marked reachable.

Best Data Structure to use is Hash Set. It stores visited hashes for O(1) lookup.

Branches often share history. Even with 50 branches, reachable commit traversal is likely near:100,000 commits not 5 million.

Then plus trees/blobs referenced by snapshots.

Could be several hundred thousand to millions of objects depending repository size.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

GC may run while commit operation is still creating new objects hence dangerous.

Race Condition where GC could delete object is Commit Process

1. Write blob object
2. Write tree object
3. Write commit object
4. Update branch ref

Before step 4 finishes, branch still points to old commit. New objects are not yet reachable from any branch. At Same Time GC Runs by scaning refs. It sees only old branch heads. New blob/tree/commit appear unreachable so deletes them.

Branch ref is then updated to new commit hash. But referenced objects were deleted so repository becomes corrupted.

How Real Git Avoids This-

Git uses protections like:

1. Locks: Ref updates and GC coordination.

2. Grace Period: Very new loose objects are not immediately deleted.

3. Reflogs: Recent refs remembered.

4. Temporary Packfiles / Atomic Updates: Objects become visible safely.

5. Stop-the-world / Safe Windows: GC usually runs when repo is idle.

GitHub Repository Link

https://github.com/harshithaha12/PES2UG24AM064-pes-vcs